ToolFunnel User Manual

Version 0.6.0

The complete guide to installing, configuring, and extending ToolFunnel: a zero-dependency MCP gateway that hosts your own tools, forwards other MCP servers, keeps the agent's context lean, and gates every call through your own policy hooks.

- Repository: https://github.com/Rendeverance/toolfunnel
- npm: https://www.npmjs.com/package/toolfunnel
- Listing: https://glama.ai/mcp/servers/Rendeverance/toolfunnel
- Licence: MIT

Contents

1. Introduction

ToolFunnel is one small MCP server that sits between your AI agent and everything else. It does five jobs at once:

- **Hosts your own tools.** Any script or command, in any language, becomes an MCP tool from one JSON entry. No SDK, no protocol code.
- **Forwards other MCP servers.** Attach an upstream server and curate which of its tools your agent sees.
- **Keeps context lean.** The agent sees short tool briefs instead of full schemas; a tool's full instructions load only when the agent reaches for it. Token cost stays flat as your toolbox grows.
- **Gates every call.** Your PreToolUse and PostToolUse policy hooks run inside the gateway, on every server-side execution path, on any client. The gate fails closed.
- **Builds you an MCP server, with no code and no SDK.** Assemble scripts and curated upstream tools, switch the meta-tools off, and ToolFunnel *is* your MCP server; `tf_pack` ships it as a `npx`-installable npm package. Section 6 (writing tools) and section 14 (packaging) cover it end to end.

It has **no runtime npm dependencies**. The MCP wire protocol is hand-rolled on Node built-ins, so `npm install toolfunnel` pulls nothing, and there is no transitive dependency tree to audit in the one component that decides what your agent may run.

This manual is the reference. If you want the short pitch, the comparison table, and the reasoning behind the zero-dependency stance, that lives in the README. Here we do the practical work: install it, wire it up, write tools for it, attach servers to it, write policy for it, and package the result.

A note on the examples: everything shown in this manual is real output from a real gateway. Where you see a screenshot, that's what you'll see; where you see JSON, that's the actual shape the files take.

Building your own MCP server

Point ToolFunnel at a few scripts in any language, or curate a handful of tools from other MCP servers, or mix the two; switch its own four meta-tools off; and what your agent connects to is a plain, top-level MCP server presenting exactly the tools you chose. No decorators, no framework, no `@tool` boilerplate, no Python. Then one `tf_pack` call turns the whole thing into a publishable npm package your users install with `npx your-mcp`.

That's a different route to the same destination a framework like FastMCP reaches: FastMCP has you write Python and decorate your functions; ToolFunnel has you declare tools in JSON (or lets the AI author them for you), and hands you a gated, zero-dependency MCP server that can also fold in *other* people's MCP servers. If you can write a script, you can ship an MCP. The mechanics are in sections 6 (writing tools) and 12 (packaging).

What's new in 0.6.0

- **Dual-era protocol support** - both eras, both roles (server to clients, client to upstreams).
- **`toolfunnel wrap`** - transparent, era-bridging single-server wrap (section 9).
- **Elicitation bridging** - mid-call user questions translated between eras (section 9.4).
- **Subscription resilience** - resource subscriptions replayed across reconnects and reloads.
- **Cancel translation** - client cancels reach upstreams in their own id space.
- **Client identity settings + wrap mirroring** (section 15.1).
- **`legacyPin`** - per-upstream legacy pinning with loud warnings (section 8.1).
- **Discover-before-enable** (section 8.2).
- Modern-era conformance details: `server/discover`, `subscriptions/listen`,

`resultType`/`ttlMs`/`cacheScope`, strict header validation, spec error-code pairing.

2. Installation and first run

You need **Node 18 or newer**. Nothing else.

From a clone

```
git clone https://github.com/Rendeverance/toolfunnel.git
cd toolfunnel
npm install    # pulls dev/test tooling only; runtime dependencies: none
```

From npm

```
npm install toolfunnel    # zero packages beyond toolfunnel itself
```

The three ways to run it

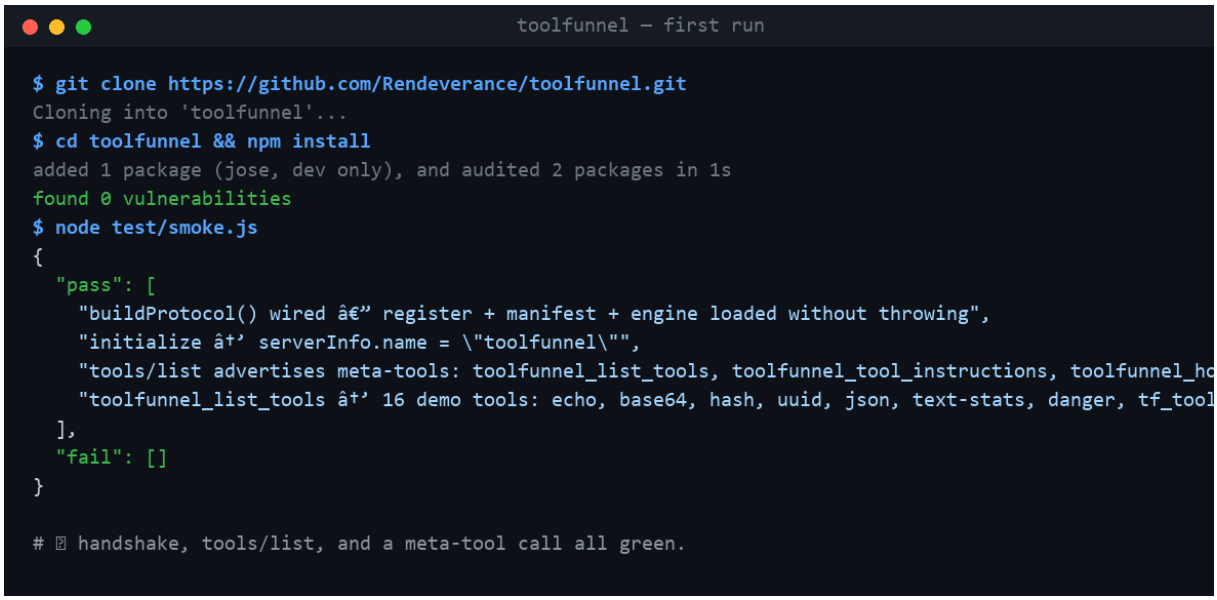
```
# 1. A stdio MCP server. This is the default, and what most MCP clients spawn.
node bin/toolfunnel.js

# 2. A long-lived HTTP host on 127.0.0.1:9998. Many clients can connect to it.
node bin/toolfunnel.js --http
node bin/toolfunnel.js --http --port 0    # 0 = let the OS assign a port

# 3. The optional config web UI on 127.0.0.1:9777 (see section 5).
node bin/toolfunnel.js --ui
```

`node bin/toolfunnel.js --help` prints the full flag list.

Prove the wiring

The smoke test builds the server, runs the MCP handshake, lists the tools, and calls a meta-tool, end to end:

```
node test/smoke.js
```



First run and smoke test

If that passes, the gateway works. The full offline suite is `npm test`.

Where things live

ToolFunnel keeps a hard split between **the engine** (`src/`, which you never edit) and **what you manage** (plain config and scripts at the top level):

```
toolfunnel/
├─ bin/toolfunnel.js        # entry point: stdio by default, --http, --ui
├─ src/                     # the engine (yours to read, not to edit)
├─ tools/                   # YOUR tools
│  ├─ tools.register.json   #   the tool register
│  ├─ tools.state.json      #   visibility overlay (enabled / hot / hidden)
│  └─ scripts/              #   the tool scripts
├─ mcp/                     # YOUR upstreams
│  ├─ expose.json           #   upstream servers + curated tools (empty by default)
│  └─ expose.example.json   #   an annotated sample
├─ hooks/                   # YOUR policy gate
│  ├─ hooks.manifest.json   #   the gate (two disabled examples ship in the box)
│  ├─ hooks.state.json      #   live enable/disable overlay
│  └─ scripts/              #   hook scripts
└─ logs/                    # audit log, created only if you enable logging
```

By default these resolve against the repo root. To keep your configuration somewhere else entirely, point the gateway at a **config home** with `--config-dir <path>` or the `TOOLFUNNEL_HOME` environment variable. An empty home is seeded from the shipped defaults on first use and never overwritten afterwards, so an `npm update` can never eat your tools. Every start prints the resolved config home to stderr — with a relocation hint when it defaulted — so there is never any doubt about where your configuration lives. Packaging (section 14) builds on this.

3. Connecting a client

Any MCP-aware client works. The two common shapes, in `.mcp.json`:

stdio (the client spawns the gateway per session):

```
{
  "mcpServers": {
    "toolfunnel": {
      "command": "node",
      "args": ["/path/to/toolfunnel/bin/toolfunnel.js"]
    }
  }
}
```

HTTP (start the host first with `node bin/toolfunnel.js --http`, then point the client at the endpoint):

```
{
  "mcpServers": {
    "toolfunnel": {
      "type": "http",
      "url": "http://127.0.0.1:9998/mcp"
    }
```

```
    }
  }
```

Once connected, the agent has four meta-tools (section 4). A sensible first conversation:

19. `toolfunnel_list_tools {}` shows the briefs of everything enabled.
20. `toolfunnel_tool_instructions { "name": "hash" }` fetches one tool's full usage.
21. `toolfunnel_run_tool { "name": "hash", "args": { "text": "hello" } }` runs it through the gate.

Or skip the JSON entirely and ask your AI in plain language: *"attach this tool server, expose these tools, and block anything destructive."* ToolFunnel ships its own built-in instructions (`toolfunnel_howto`), so an MCP-aware agent can wire up upstreams, exposure, and a safety gate for you. No coding experience required :)

4. How ToolFunnel thinks

Five ideas carry the whole design. Ten minutes here saves you an hour everywhere else.

4.1 The meta-tool surface

The model never sees your long tail of tools directly. It sees four fixed **meta-tools**:

Meta-tool	Args	Returns
`toolfunnel_list_tools`	`{ filter?, category? }`	Briefs only: `[{ id, name, summary, category }]`
`toolfunnel_tool_instructions`	`{ name }`	One tool's full usage instructions, on demand
`toolfunnel_howto`	`{ topic }`	A self-extension guide: `create-tool`, `add-mcp`, `add-hook`, `package`
`toolfunnel_run_tool`	`{ name, args }`	Runs a register tool through the gate

This is the token saver. A conventional MCP server injects every tool's full JSON schema into the model's context on every turn. ToolFunnel advertises four small schemas, and the agent pulls one tool's detail only when it actually wants that tool.

4.2 The visibility matrix

Every tool, including the four meta-tools themselves, has three independent dials in `tools/tools.state.json`:

Dial	What it controls	Default
`enabled`	Lean-visible: appears in `toolfunnel_list_tools` and is runnable	on
`hot`	Promoted to the top-level `tools/list` surface: full schema on every turn, directly callable	off (meta-tools: on)

	without the `toolfunnel_run_tool` hop	
`hidden`	Declutters the manager views (`tf_list`, the UI) only; does not change what the agent sees	Off

The dials are read fresh on every call, so a toggle is live with no restart. A disabled tool is never hot.

The matrix is what lets ToolFunnel be a lean register and a conventional MCP at the same time. Three postures:

- **Lean (default).** Four meta-tools top-level, everything else briefs-on-demand.
- **Mixed.** Flip `hot` on the handful of tools you call constantly; they join the every-turn surface while the long tail stays lean. A promoted call still passes the gate.
- **Plain MCP.** Promote your chosen set and switch the meta-tools off (`hot:false`). ToolFunnel now presents exactly those tools as an ordinary top-level MCP server, assembled from scripts and other MCPs, with no SDK. A de-promoted meta-tool (`hot:false`) is also un-callable, not merely unlisted — the lockdown is real, not cosmetic.

Two footguns, which the UI warns about: hiding the meta-tools removes the agent's ability to discover tools by name (intended for the plain-MCP posture, a mistake otherwise), and promoting many tools reintroduces exactly the context bloat the lean register exists to avoid.

4.3 Execution modes: reference vs gateway

Each register tool resolves to one of two modes (an explicit `mode` field, or inferred from the entry):

- **`reference`**: ToolFunnel only *describes* the tool. `toolfunnel_run_tool` hands back the instructions and the connected AI performs the action in its own environment. Nothing runs server-side. The handoff itself is still gated: a PreToolUse deny withholds the instructions.
- **`gateway`**: ToolFunnel *executes* the tool server-side, through the full gated run path.

When `mode` is omitted: a `script` or `shell` invoke infers `gateway`; anything else infers `reference`. The resolved mode is shown, and switchable, per tool in the UI.

4.4 The gate

Every server-side run path funnels through one function:

```
gatedRun({ engine, ctx, toolName, args, execute })
  → fire PreToolUse   (may BLOCK; fails CLOSED if the engine errors)
  → execute()         (ONLY if allowed)
  → fire PostToolUse  (advisory; cannot un-run the tool)
```

That includes your own gateway tools, all nine management functions, and every forwarded upstream call. The invariant, proven by test: **a PreToolUse deny means `execute()` is never called.** Section 10 covers writing hooks.

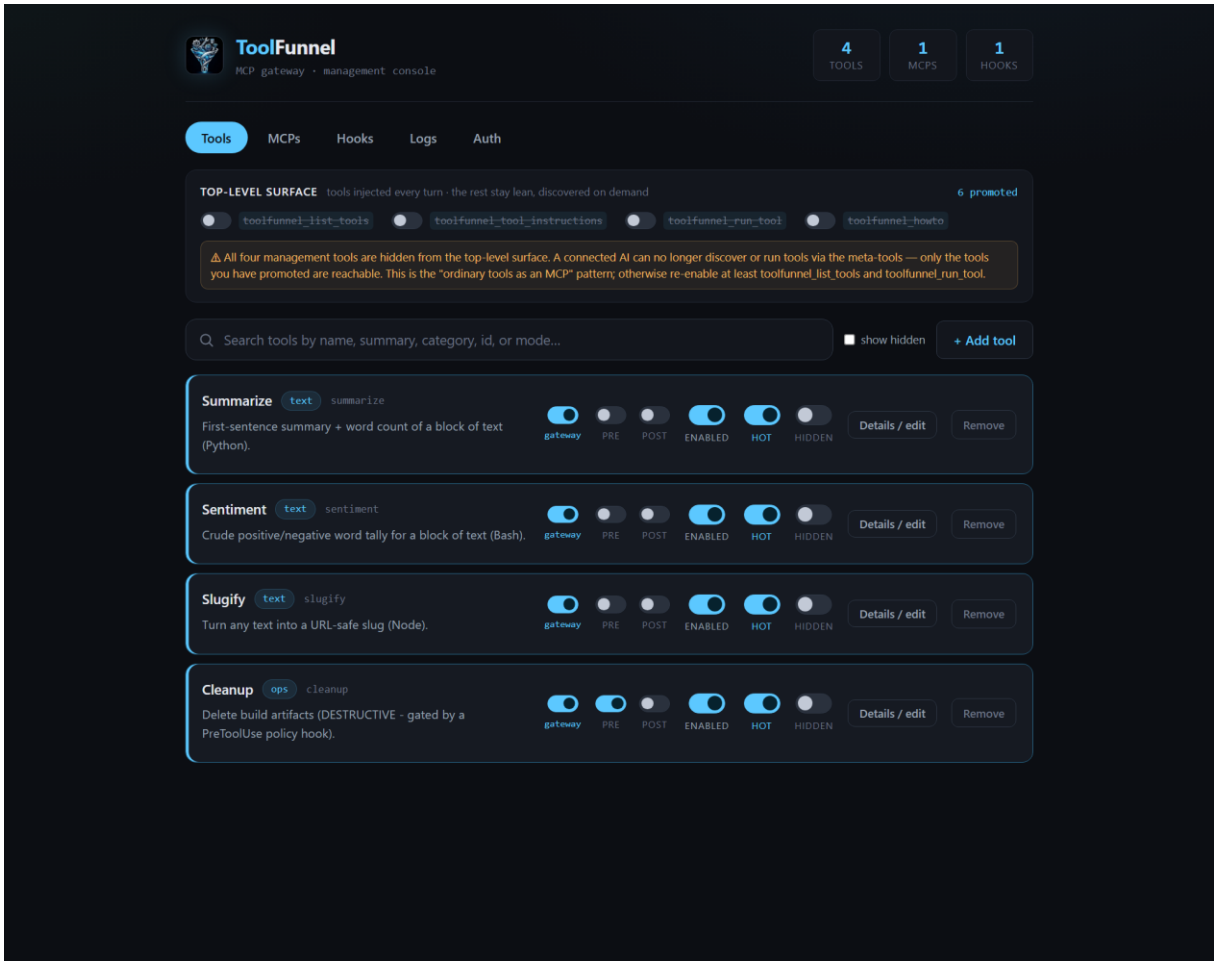
4.5 Live everything

Tool register edits, state toggles, hook changes, and upstream attach/curate changes are all picked up on a running gateway: config is re-read per call or watched, upstreams reconnect, and the gateway emits `notifications/tools/list_changed` so a connected client refreshes its view. If an attached upstream process dies, the gateway reconnects it in the background with backoff. You should not need to restart ToolFunnel in normal use.

5. The config web UI

```
node bin/toolfunnel.js --ui # http://127.0.0.1:9777
```

The UI is optional, loopback-only by design (it refuses any non-loopback bind, because it can spawn processes and write scripts), and dependency-free: vanilla HTML/CSS/JS served by the gateway itself, no CDN, works offline. Every write goes through the same stores the MCP server reads, so a UI edit is byte-identical to a hand edit and is live immediately.

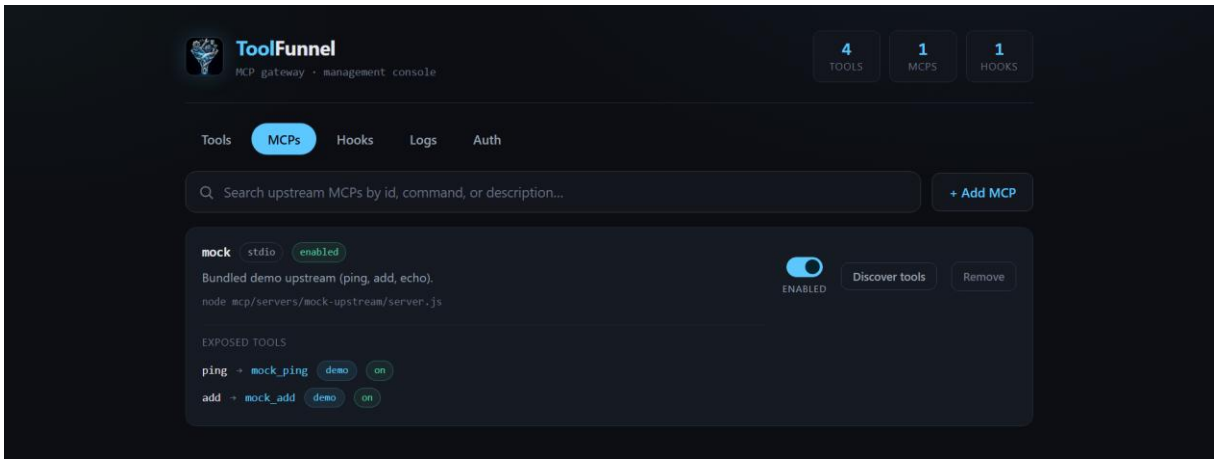


The Tools tab

Five tabs:

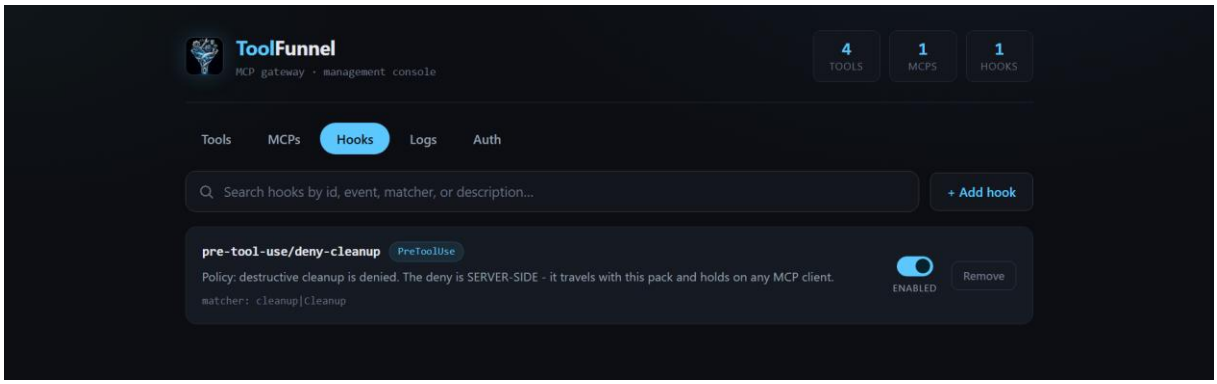
Tools. Every register tool with live search and a show-hidden filter. Per tool: toggle **Enabled**, **Hot**, and **Hidden**, flip the execution mode (reference ↔ gateway), attach per-tool Pre/Post hook gates, or remove it. The top-level **surface panel** shows the four meta-tools' every-turn toggles, a promotion count, and the footgun warnings from section 4.2. The Add form registers a new tool, optionally authoring its script body straight into `tools/scripts/`.

MCPs. Your upstream servers and their curated expose selections: add an upstream, enable, disable, remove. The **Discover** button does a live connect-and-list of an upstream's tools, each with its own lean and Hot toggle, so you can curate by ticking rather than by editing JSON.



The MCPs tab

Hooks. The hook manifest with live enabled state. The two shipped example hooks (section 10) appear here, disabled; one click arms them. Add a lifecycle hook, optionally authoring its script body.



The Hooks tab

Logs. The audit log's on/off switch and a newest-first view of recent records (section 12).

Auth. The optional OAuth 2.1 resource-server configuration, plus a one-click install of its single dependency (section 13).

6. Your own tools

This is the point of the thing: any script you've already got becomes a gated MCP tool from one JSON entry.

6.1 The register entry

Tools are declared in `tools/tools.register.json`. A complete entry:

```
{
  "id": "hash",
  "name": "Hash",
  "summary": "Compute a cryptographic digest (e.g. sha256) of input text.",
  "category": "crypto",
  "instructions": "Args: { algo?: \"sha256\"|\"sha1\"|\"md5\" (default \"sha256\"),
text: string }. Returns the lowercase hex digest of the UTF-8 text.",
  "invoke": {
    "type": "script",
    "path": "scripts/hash.js"
  }
}
```

- `summary` is what the agent sees in the lean brief. Keep it one line.
- `instructions` is what `toolfunnel_tool_instructions` returns. Document the args here the way you would want them documented for you.
- `invoke` is what runs. `{ "type": "script", "path": "scripts/x.js" }` spawns node on the script; a `shell` invoke runs a command line; omit `invoke` entirely for a reference tool (section 4.3).
- An optional `inputSchema` (JSON Schema) is advertised verbatim when the tool is promoted hot, so strict clients see your real schema.

6.2 The script contract

A gateway tool script is a standalone program with a three-line contract:

22. Read your structured args from the environment variable `TOOLFUNNEL_TOOL_ARGS` (a JSON string).
23. Print **exactly one JSON line** to stdout.
24. Exit `0` on success.

The smallest possible tool:

```
// tools/scripts/shout.js
const args = JSON.parse(process.env.TOOLFUNNEL_TOOL_ARGS || "{}");
```

```
console.log(JSON.stringify({ shouted: String(args.text || "").toUpperCase() }));
```

The spawn happens only after the PreToolUse gate allows it. The contract is language-agnostic in spirit: a `shell` invoke can point at Python, bash, a compiled binary, anything that can read an environment variable and print a line of JSON.

6.3 Adding a tool, three ways

All three are equivalent and live immediately.

By hand: add the entry to `tools/tools.register.json`, drop the script in `tools/scripts/`.

Through the UI: Tools tab → Add. The form takes the register fields and, optionally, the script body itself.

In-band, by the agent: the `tf_tool_add` management function registers a tool, and can author the script too:

```
tf_tool_add {
  "id": "shout",
  "name": "Shout",
  "summary": "Uppercase the provided text.",
  "category": "text",
  "instructions": "Args: { text: string }. Returns { shouted }.",
  "invoke": { "type": "script", "path": "scripts/shout.js" },
  "scriptText": "const args = JSON.parse(process.env.TOOLFUNNEL_TOOL_ARGS ||
\"{}\");\nconsole.log(JSON.stringify({ shouted: String(args.text ||
\"\").toUpperCase() }));"
}
```

Script authoring is path-guarded: a script lands under `tools/scripts/` and nowhere else.

6.4 The demo tools

Seven first-party demos ship in the box so a fresh install has something to poke: `echo`, `base64`, `hash`, `uuid`, `json`, `text-stats`, and `danger`. The last one is a deliberately "destructive" demo whose only side effect is appending a line to the file named by `TOOLFUNNEL_DANGER_LOG`; it exists so you can watch the gate block something (section 10.3). They are just there to get you started. Turn them off or delete them as you like :)

7. Attaching other MCP servers

7.1 The shape

Upstreams and their curated exposure live in `mcp/expose.json`. It ships empty (the gateway connects to nothing until you say so). A worked example, wiring the bundled zero-dependency demo upstream and exposing two of its three tools:

```
{
  "version": 1,
  "upstreams": [
    {
      "id": "mock",
      "transport": "stdio",
      "command": "node",
      "args": ["mcp/servers/mock-upstream/server.js"],
      "enabled": true,
      "description": "Bundled demo upstream MCP: ping, add, echo."
    }
  ],
  "expose": [
    { "upstream": "mock", "tool": "ping", "as": "mock_ping", "category": "demo",
"enabled": true },
    { "upstream": "mock", "tool": "add",  "as": "mock_add",  "category": "demo",
"enabled": true }
  ]
}
```

The same file is in the box as `mcp/expose.example.json` with a longer commentary.

- Each `expose` entry curates one upstream tool into your surface, under the surfaced name `as` (or `<upstream>_<tool>` if you omit it).
- Everything you do NOT expose stays invisible. Curation is the default posture, not an afterthought.
- An exposed upstream tool obeys the same visibility matrix as your own tools, keyed by its surfaced name, and every forwarded call passes the gate.

One guard worth knowing about: the aggregator's **isolation guard** rejects any path-shaped upstream arg that escapes the gateway root. A vendored upstream must live inside the tree; the interpreter slot (`node`, `python`, `npx`) is exempt. This stops a config edit quietly pointing the gateway at something outside its sandbox.

7.2 Discover and curate by clicking

The UI's MCPs tab has a **Discover** button per upstream: a live connect-and-list of everything the upstream offers, each tool with its own lean and Hot toggle. Tick what you want, done. `tf_mcp_add` does the same in-band:

```
tf_mcp_add {
  "id": "search",
  "command": "npx",
  "args": ["-y", "@modelcontextprotocol/server-brave-search"],
  "env": { "BRAVE_API_KEY": "..." },
  "expose": [ { "tool": "brave_web_search", "as": "web_search" } ]
}
```

7.3 Health and self-healing

If an attached upstream's process dies, the gateway notices, reconnects in the background with backoff (fast attempts first, then a slow patrol), and re-advertises its tools when it returns. No restart, no dropped session. Connection state is visible on the MCPs tab and in the `/health` snapshot when running the HTTP host.

8. The two protocol eras

On 28 July 2026 the Model Context Protocol changed in a breaking way. The revision removed the `initialize` handshake, protocol-level sessions, and the standalone SSE endpoint that every earlier client and server was built on, replacing them with per-request metadata, a `server/discover` endpoint, and a `subscriptions/listen` notification stream.
In this manual we call everything before that revision the **legacy era** and the revision itself the **modern era**. The practical consequence: a modern client cannot talk to a legacy server, and a legacy client cannot talk to a modern server — *unless something in the middle speaks both*.
ToolFunnel 0.6.0 speaks both, in both roles:

- **As a server** it answers legacy clients (the classic handshake) and modern clients (per-request metadata) on the same endpoint at the same time. No mode switch — each request is served in the era it arrives in.
- **As a client** it probes each attached server (`server/discover` first, then the legacy handshake — the spec's own order) and speaks whichever era that server understands.

You don't configure any of this by default. It is how the gateway works now. Two optional switches narrow it when you want a smaller attack surface: `"serveLegacy": false` in `toolfunnel.json` makes the gateway refuse legacy-era clients entirely (a clear error naming the policy, a loud startup warning, and the Settings tab has the toggle — only an explicit `false` flips it, so a misconfigured file can never lock clients out), and `"modernOnly": true` on an upstream makes the gateway refuse to speak legacy TO that server (the connect fails cleanly instead of negotiating down — the mirror of `legacyPin`, and setting both on one upstream is refused). Every current mainstream client speaks the legacy era: leave `serveLegacy` alone until your clients have moved.

8.1 The legacy shim (`legacyPin`)

Separate from wrapping: any attached upstream can be pinned to the legacy protocol with `"legacyPin": true` in its `expose.json` entry. A pinned server is spoken to as 2024-11-05 unconditionally — the modern probe is skipped entirely. Off by default; the gateway warns at startup and on every forwarded call so a pin is never silent. Use it for a server that misbehaves when probed, or to freeze behaviour while debugging.

8.2 Inspecting before enabling

"Discover tools" works on a **disabled** upstream — you can inspect what a server offers before switching it on. The disabled server still never appears on any tool surface.

8.3 Per-upstream tuning (`env`, `cwd`, `timeoutMs`)

Three optional fields on an `upstreams[]` entry tune how its server is run:

- `"env"` — extra environment variables for the spawned process.

- `"cwd"` — its working directory.
- `"timeoutMs"` — how long a tool call on this upstream may run (default 120000 ms; applies to tool calls, prompt gets and resource reads). A tool that reports progress keeps its call alive regardless of this limit, so it only matters for tools that run long *silently*. The short 10-second window used elsewhere is deliberate and not configurable: it applies only to handshakes and listings, where a slow answer means a dead server.
- `"modernOnly"` — require this upstream to speak the modern (2026-07-28) protocol. The connect fails with a clear error instead of falling back to legacy. The mirror of `legacyPin` (section 8.1); setting both on one upstream is refused.

9. Wrapping an MCP server

9.1 What wrapping is

Normally ToolFunnel is a *funnel*: many tools and servers behind one lean surface, with its own meta-tools on top. **Wrapping inverts that.** One command:

```
toolfunnel wrap <upstreamId>
```

…and ToolFunnel *becomes* that one server. Its tools are the entire advertised surface, under their original names. The meta-tools disappear and become uncallable. The wrapped server's own name, version, description and instructions are what a connecting client sees. `toolfunnel wrap --off` restores the normal funnel. A running gateway picks up either change live — no restart.
[SCREENSHOT: the UI wrap banner + wrap/unwrap buttons]

9.2 Why you would wrap

- **Keep a legacy server alive past the cutover.** Your favourite unmaintained MCP server will stop working when your client upgrades to the modern protocol. Wrap it, and modern clients use it as if it had been upgraded — because from the outside, it has been.
- **Give a modern-only server to older tooling.** The same bridge runs in reverse.
- **Put a policy gate in front of a server you didn't write** — every call through the wrap can still fire your PreToolUse hooks, without the client or the server knowing the gate is there.

All four era combinations work: legacy client → legacy server, legacy → modern, modern → legacy, modern → modern. Zero configuration beyond attaching the server and typing `wrap`.

9.3 Invisibility — what the client sees, what the server sees

A wrap is designed to be **undetectable from both sides**.
The client sees the wrapped server's identity — name, title, version, instructions, capabilities — exactly as the server reports them. Tool definitions pass through byte-for-byte, including descriptions, annotations and any fields the spec adds in future. Results, errors (codes, messages, data — verbatim), notifications, progress updates and resource subscriptions all pass through unmodified. This is verified by automated tests that compare a direct connection against a wrapped one, byte for byte, against real published servers.
The server sees a normal client. On stdio, ToolFunnel **mirrors your real client's identity**

upstream — the wrapped server sees the same `clientInfo` it would see if your client connected directly. (Over HTTP, where many clients share one gateway, the configured client identity is presented instead — see section 15.1.)
The one deliberate exception: the era translation itself. A legacy server wrapped for a modern client is, by definition, receiving legacy-era requests from the gateway — that is the feature.

9.4 Mid-call questions (elicitation) survive the wrap

Some servers ask the user a question in the middle of a tool call — "which file did you mean?", "confirm delete?" — a feature called *elicitation*. The two eras do this incompatibly: legacy servers hold the call open and send the question backwards over the connection; modern clients instead receive an `input_required` result and *retry* the call with the answers attached.
The wrap translates between them. When a wrapped legacy server asks a question mid-call:

- a **modern client** receives a spec-correct `input_required` result carrying the question and a resume token; when it retries with the answers, the gateway feeds them to the server's held call, which completes normally. Multi-round questioning works; tokens are single-use; an abandoned question is cancelled after 10 minutes so nothing leaks.
- a **legacy client** (which cannot receive backwards questions through the wrap yet) has the question automatically declined, and the call completes with the server's declined-path result — degraded, but never hung.

This is tested end-to-end against real published elicitation servers, not just fixtures.

9.5 Robustness details you get for free

- **Crash healing:** if the wrapped server's process dies, the gateway reconnects it in the background with backoff — and *re-establishes any resource subscriptions* your client had, so update notifications resume without the client ever knowing.
- **Cancellation:** a client's cancel is translated into the server's own request-id space — including cancels of long-running tool calls — so the server actually stops work instead of running abandoned calls to completion.
- **Config reloads** (attaching/removing other servers, hook edits) never interrupt the wrap.

9.6 Wrapping & security

In funnel mode, ToolFunnel applies a **path-isolation guard** to every attached server: file paths in a server's arguments must live inside ToolFunnel's own directory, so a config edit can't quietly point the gateway at code or data elsewhere on your machine.
Wrapping suspends that guard — for the wrapped server only, and only while it is wrapped. That is deliberate. Wrapping is an explicit *"this server IS my tool surface"* declaration, and wrapped servers routinely need outside paths to be useful at all: a filesystem server pointed at your documents folder, a database server pointed at a `.db` file, a server installed by absolute path. The `wrap` command tells you, loudly, when this applies:

```
[toolfunnel] ⚠ WRAP SECURITY NOTICE — upstream "myfs" uses paths outside the gateway
root: …
```

What still protects you while wrapped:

- **Every call still passes the PreToolUse gate.** A hook that returns a block stops the call before it reaches the server.
- **Per-tool switches still work.** Disable a tool and it is neither advertised nor callable — the switch is keyed by the tool's surfaced name (`<upstreamId>_<toolName>`).
- **Other attached servers keep the guard.** The exemption follows the one wrapped server.

Restricting a wrapped server, worked example — a filesystem server you want read-only:

25. **Block by tool with a PreToolUse hook** — deny `write_file`, `edit_file`, `move_file`, `delete_*` (hooks match tool names and can inspect arguments; see the Hooks chapter).
26. **Or disable tools outright** — `tf_tool_set myfs_write_file enabled:false`, or untick it in the UI matrix. Disabled tools vanish from the wrapped surface.
27. **Check the result** — `tools/list` through the wrap shows exactly what a client can see and call. What is advertised is what is callable.

Rules of thumb: wrap servers you trust with the paths you gave them — the wrap is a megaphone, not a cage; if you want a cage, put the gate in front of the risky tools before handing the wrapped gateway to a client; and read the security notice — it lists the exact paths in play. `wrap --off` restores the guard at the next connection.

10. The policy gate

The gate is the reason ToolFunnel exists rather than being a clever proxy. Policy lives **in the gateway**, so it applies on any client, and it **fails closed**.

10.1 The manifest

Hooks are declared in `hooks/hooks.manifest.json`. An empty `hooks` array means allow-all. The shipped manifest carries two disabled examples; here's the PreToolUse one (its `description` field abridged for space):

```
{
  "id": "pre-tool-use/example-deny-dangerous",
  "event": "PreToolUse",
  "matcher": "*",
  "type": "command",
  "command": "node \"${HOOKS_DIR}/scripts/example-deny-dangerous.js\"",
  "script": "scripts/example-deny-dangerous.js",
  "timeout": 10,
  "enabled": false,
  "description": "Denies any tool call whose arguments contain an obviously-
destructive shell pattern."
}
```

- `event`: `PreToolUse` (can block) or `PostToolUse` (advisory, runs after).
- `matcher`: a pattern matched against the tool's surfaced name. `*` is everything; `tf_.*` locks down the whole management surface in one line. Matching is full-anchored, so a gate on `read` does not leak onto `read_file`.

- `command`: what runs. Use the portable `${HOOKS_DIR}` token rather than an absolute path.
- `enabled` here is the manifest seed; live toggles go through the `hooks/hooks.state.json` overlay (which is what the UI writes), so you can arm and disarm without editing the manifest.

10.2 The hook protocol

ToolFunnel speaks the Claude Code hook protocol, deliberately, so an existing hook travels here unchanged:

28. The hook command receives the event as JSON on **stdin**: tool name, args, context.
29. To **block**, exit `2` (with the reason on stderr) — or exit `0` and print `{ "decision": "block", ... }` or `{ "hookSpecificOutput": { "permissionDecision": "deny" } }`.
30. Anything else allows.

A complete policy hook, small enough to read in one breath:

```
// hooks/scripts/no-hash-md5.js — deny md5 digests, allow everything else
let raw = "";
process.stdin.on("data", d => raw += d);
process.stdin.on("end", () => {
  const evt = JSON.parse(raw || "{}");
  if (evt.tool_name === "hash" && evt.tool_input && evt.tool_input.algo === "md5") {
    console.error("md5 is not a serious digest. Denied.");
    process.exit(2);
  }
  process.exit(0);
});
```

Register it with `tf_hook_add` (which can author the script body for you), the UI's Hooks tab, or by hand in the manifest.

10.3 Watch it bite

The `danger` demo tool exists for exactly this. Enable the shipped example gate (Hooks tab, one click, or `tf_hook_set { "id": "pre-tool-use/example-deny-dangerous", "action": "enable" }`), then ask the agent to run `danger` with a destructive-looking argument. The gate blocks it before anything executes, the agent receives the denial reason, and if the audit log is on, the decision is recorded. A PostToolUse hook cannot un-run a tool; it is the observation point, not the barrier.

10.4 What is actually gated

Every server-side execution: gateway tools run via `toolfunnel_run_tool`, hot-promoted local tools (their direct calls route through the same gated path), all nine `tf_*` management functions, and every forwarded upstream call. Reference tools execute nothing server-side, so what gets gated there is the instruction handoff itself.

11. The management functions

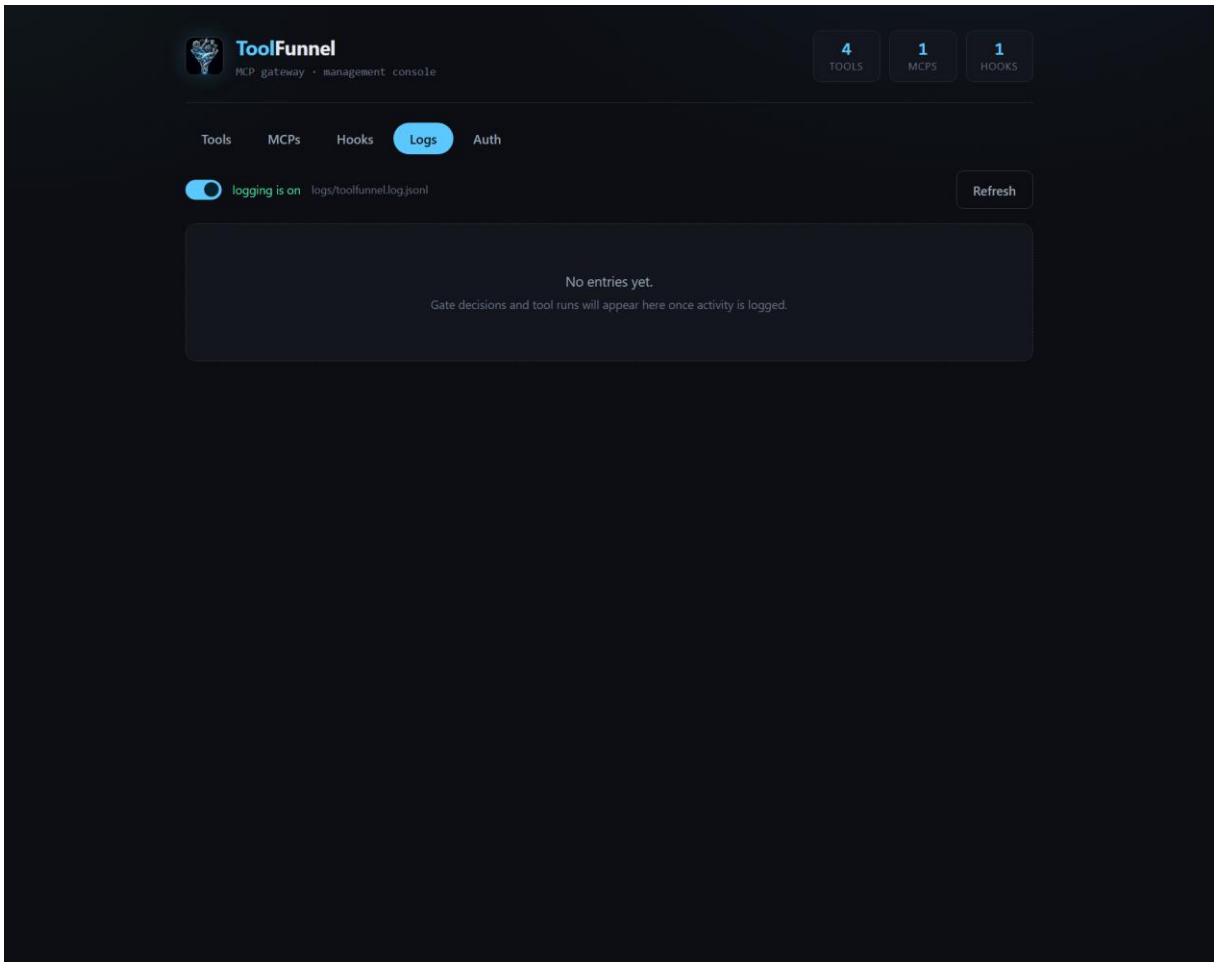
Nine first-party tools in the `management` category let an agent, or the UI, manage the gateway itself. They are ordinary register tools: reached through `toolfunnel_run_tool`, gated like everything else, and a single `tf_.*` matcher locks the lot down.

Function	Args	Notes
`tf_tool_add`	`{ id, name?, summary?, category?, instructions?, invoke?, inputSchema?, scriptText? }`	Register a tool; `scriptText` authors the script under `tools/scripts/` (path-guarded). Live.
`tf_tool_set`	`{ id, action }` where action ∈ `enable, disable, hot, unhot, hide, unhide, remove`	The visibility matrix dials, plus removal.
`tf_mcp_add`	`{ id, command, args?, env?, transport?='stdio', enabled?=true, description?, expose?: [{ tool, as?, category?, enabled? }] }`	Adds an upstream; each `expose` item curates one tool.
`tf_mcp_set`	`{ id, action: 'enable' \`	'disable' \
`tf_hook_add`	`{ id, event, matcher?, command, script?, timeout?, enabled?=true, description?, scriptText? }`	`id` format `<kebab-event>/<name>`; use `${HOOKS_DIR}` in the command.
`tf_hook_set`	`{ id, action: 'enable' \`	'disable' \
`tf_list`	`{ kind: 'tools' \`	'mcps' \
`tf_log`	`{ action: 'enable' \`	'disable' \
`tf_pack`	`{ format: 'home' \`	'npm', name?, ... }`

12. The audit log

Off by default: a fresh checkout writes nothing and creates no log file. When enabled, ToolFunnel keeps a JSONL log of tool runs, **every gate allow/deny decision**, and upstream connect / disconnect / reconnect events.

Three equivalent switches: `tf_log { "action": "enable" }`, the UI's Logs tab, or editing `logs/log.config.json` (`{ "enabled": true, "path": "logs/toolfunnel.log.jsonl" }`). Config is read fresh per event, so the toggle is immediate. Logging is wrapped so it can never throw into a tool call or the gate.



The Logs tab

13. The HTTP host and OAuth 2.1

13.1 The host

```
node bin/toolfunnel.js --http                  # 127.0.0.1:9998
node bin/toolfunnel.js --http --port 0         # OS-assigned port
```

One Streamable-HTTP endpoint at `POST/GET /mcp` (plus a deprecated `GET /mcp/sse` alias), and a `GET /health` JSON snapshot with in-memory call counters and upstream connection state. (The dual-framing tolerance — reading both LSP-style `Content-Length` and newline-delimited JSON — is a property of the *stdio* transport described in sections 2 and 3; the HTTP host is one JSON-RPC request per POST body plus the SSE stream.)

By default the host binds loopback only and rejects non-loopback `Host` headers (a DNS-rebind guard). It **refuses to bind off-localhost unless OAuth is enabled** — that refusal is deliberate and there's no override flag; auth is the key that unlocks the network. A fair warning to match the

README's candour: OAuth and the Streamable-HTTP host are the newest parts of ToolFunnel and the least exercised, so if you're deploying networked, test your setup before you lean on it.

13.2 OAuth 2.1 resource server

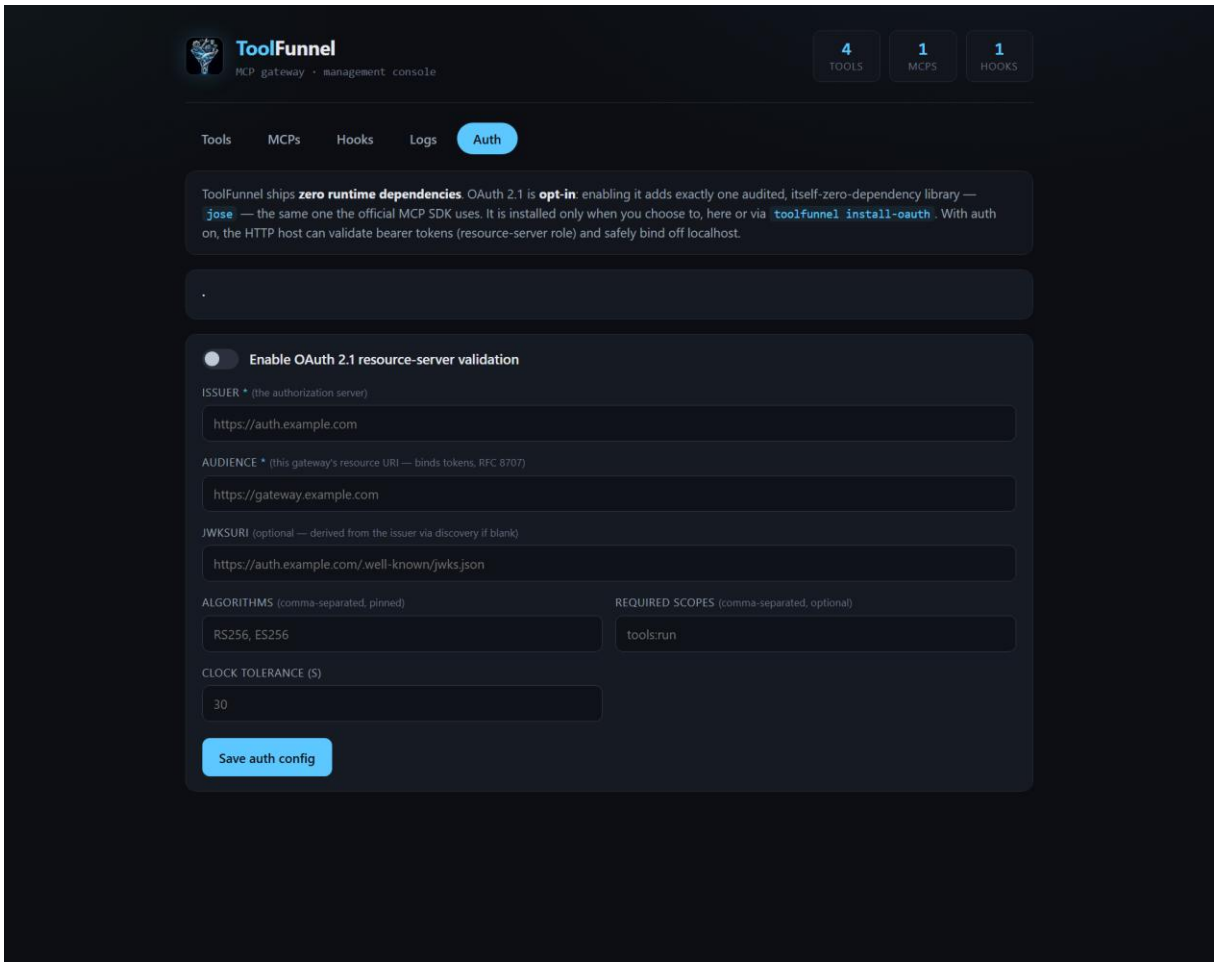
For a networked or multi-user deployment, ToolFunnel validates a bearer token on every request before any tool runs.

- **Opt-in, one dependency.** Off by default. Enabling installs `jose` on demand (`toolfunnel install-oauth`, or the Install button on the UI's Auth tab). `jose` is itself zero-dependency and is the same library the official MCP SDK uses. The core stays dependency-free for everyone who leaves auth off.
- **Validation that does the things that matter:** a pinned algorithm allowlist (which defeats RS256/HS256 confusion and `alg:none`), enforced issuer, an enforced audience bound to this gateway's resource URI (the RFC 8707 confused-deputy defence), and `exp` required. JWKS fetch, caching, and rotation are the library's job.
- **Discovery.** With auth on, the gateway serves RFC 9728 Protected Resource Metadata at `GET /.well-known/oauth-protected-resource`, and a `401` carries the `WWW-Authenticate: Bearer resource_metadata="…"` hint.

Configure on the Auth tab or in `auth/auth.config.json`:

```
{
  "enabled": true,
  "issuer": "https://auth.example.com",
  "audience": "https://mcp.example.com/toolfunnel",
  "algorithms": ["RS256"],
  "requiredScopes": [],
  "clockToleranceSec": 5
}
```

`jwksUri` is optional; omitted, it is derived from the issuer via OIDC discovery. The gateway fails fast at startup if auth is on but the dependency is missing or the config is incoherent.



The Auth tab

The OAuth **client** leg (Dynamic Client Registration, authorization-server-metadata discovery, step-up auth) is planned, not yet shipped. The resource-server slice is the shipped, tested MVP.

14. Packaging: ship your own MCP

Everything you build — tools, curation, upstream selections, policy hooks, identity — lives in one folder (the config home). One management call packages it.

14.1 A portable home

```
tf_pack { "format": "home" }
```

produces a portable config home under `dist/`: zip it, commit it, hand it to a colleague. Run it anywhere with `node bin/toolfunnel.js --config-dir <the-home>`.

14.2 A publishable npm package

```
tf_pack { "format": "npm", "name": "my-mcp" }
```

produces a publishable npm package that **depends on** toolfunnel (a caret range — it never forks or copies the engine), bundles your config home, and provides a two-line `bin` so that after you publish it, your users get:

```
npx my-mcp
```

Your server, your name in the MCP handshake, your tools and schemas — and **your gate enforced on their machine regardless of which client they use**. Depend-not-copy matters: every security fix to toolfunnel reaches every wrapped MCP through a normal `npm update`, with no stale forks.

14.3 Identity and requires

`toolfunnel.json` at the home root gives the gateway its identity and defaults:

```
{
  "serverName": "my-mcp",
  "serverVersion": "1.0.0",
  "httpPort": 9998,
  "uiPort": 9777,
  "requires": [
    { "command": "python", "min": "3.10", "why": "the pdf-extract tools" },
    { "command": "git" }
  ]
}
```

`requires` is an array of runtime checks your pack's tools need. Each entry needs a `command` (a bare program name, probed on PATH); `min` is an optional minimum version in dotted numerics (`"3.10"`, `"18"` — no range operators), `versionArg` overrides the default `--version` flag, and `why` is the human reason shown if it's missing. The gateway probes each at startup and prints a clear stderr line naming what's absent, the version found versus needed, and your `why`, instead of failing obscurely mid-tool-call. Packs are composite by nature: your own scripts, upstream references (fetched on the user's machine, pin versions in the pack), and curation all travel together.

One honesty note that belongs in every packaging story: packs spawn commands. Read the `expose.json` and the register of anything you install, the same way you would read a shell script before running it.

The full packaging walkthrough lives in `docs/packaging.md` in the repository.

15. Configuration reference

All configuration is plain JSON. Edit by hand, through the UI, or in-band; all three write the same stores.

File	Purpose
`tools/tools.register.json`	The tool register: `{ version, description, tools: [...] }`. Seven demos + nine management functions ship in it. Entries may carry an `inputSchema`, advertised verbatim when hot.
`tools/tools.state.json`	The visibility overlay, keyed by surfaced name → `{ enabled?, hot?, hidden? }`. Empty `{}` means everything enabled, nothing hot (except the meta-tools), nothing hidden. Read fresh per call.
`mcp/expose.json`	Upstream servers + curated exposure: `{ version, upstreams: [], expose: [] }`. Empty by default: the gateway connects to nothing until told.
`mcp/expose.example.json`	The annotated sample from section 7.1.
`hooks/hooks.manifest.json`	The policy gate: `{ version, hooks: [] }`. Empty array = allow-all. Two disabled examples ship.
`hooks/hooks.state.json`	Live enable/disable overlay for hooks; an entry here wins over the manifest seed.
`logs/log.config.json`	The audit-log switch: `{ enabled, path }`. Default off; created only when logging is first enabled.
`toolfunnel.json`	Optional identity + defaults: `{ serverName, serverVersion, httpPort, uiPort, requires }`. Absent = the gateway introduces itself as plain `toolfunnel`.
`auth/auth.config.json`	The OAuth resource-server config from section 13.2. Absent or `enabled:false` = unauthenticated loopback.

Flags and environment: `--http`, `--ui`, `--port`, `--host`, `--config-dir <path>` / `TOOLFUNNEL_HOME`. `node bin/toolfunnel.js --help` is the authoritative list.

15.1 Identity settings (`toolfunnel.json`)

The optional `toolfunnel.json` in the config home now controls both directions of identity:

```
{
  "serverName":    "my-mcp",      // what CLIENTS see in the funnel's own handshake
  "serverVersion": "1.0.0",
  "clientName":    "my-client",   // what UPSTREAM SERVERS see from the gateway's
client
  "clientVersion": "1.0.0",
  "httpPort":      9998,
  "uiPort":        9777
}
```

- `serverName`/`serverVersion` — the funnel's own identity (unchanged from 0.5.0; a wrap ignores these and presents the wrapped server instead).
- `clientName`/`clientVersion` — **new in 0.6.0**: the identity the gateway presents to upstream

servers. Defaults to `toolfunnel`. Under a wrap on stdio this is overridden by the mirrored identity of your real client (section 9.3); in funnel mode and over HTTP it is what upstreams see. Every field is optional; a missing or broken file falls back field-by-field to the defaults.

15.2 A complete gateway from five files (no code)

Everything above is plain JSON — there is nothing to program. A gateway that attaches one upstream, exposes two of its tools, gates everything, logs activity, and introduces itself as `acme-tools`:

31. `toolfunnel.json` — `{ "serverName": "acme-tools", "clientName": "acme-tools" }`
32. `mcp/expose.json` — one `upstreams[]` entry (id, `"transport": "stdio"`, command, args, `"enabled": true`) plus two `expose[]` entries (`{ "upstream", "tool", "enabled": true }`)
33. `hooks/hooks.manifest.json` — one PreToolUse entry matching the tools you want policy on
34. `logs/log.config.json` — `{ "enabled": true }`
35. Start it: `node bin/toolfunnel.js` (stdio) or `--http` / `--ui` as needed.

Every file is read fresh per request, so later edits are live without a restart — `toolfunnel.json` (identity and ports) is the one exception: it is read at boot, so restart to apply. An agent can fetch this same map in-band with `toolfunnel_howto({ "topic": "configure" })`.

16. Design decisions

Short notes on the decisions that shape ToolFunnel, so you can judge the reasoning without reading the source. The same notes ship in the repository as `docs/design.md`.

Zero runtime dependencies. ToolFunnel sits in a privileged position: it executes tools and enforces policy. Every transitive dependency in that position is something you have to trust or audit. So the gateway uses Node built-ins only; `"dependencies"` in package.json is empty and stays empty. The one exception is deliberate and opt-in: OAuth token validation uses jose, installed only when you run `toolfunnel install-oauth`, version-pinned, because hand-rolled JWT crypto is how security holes are born. Dev tooling for the test suite never ships.

No SDK. The gateway hand-rolls JSON-RPC over stdio and HTTP rather than building on the official MCP SDK. Two reasons. First, the zero-dependency stance above. Second, ToolFunnel has to speak two protocol eras at once, on both sides, which the SDK does not do; owning the wire layer is what makes the dual-era serving and the wrap possible at all.

The gate fails closed. Every server-side execution path goes through one function, and that function treats every failure as a deny: a hook that blocks, a hook engine that crashes, a malformed engine result, all of them mean the tool does not run. Denies carry a flag distinguishing operator policy ("your hook said no") from wiring failure ("the gate itself broke"), so a broken gate can never be mistaken for an approving one. The invariant has its own test.

The isolation boundary. Upstream server definitions can name any executable but their path-shaped arguments must stay inside the config home. The command is the interpreter you chose; the args are what it can reach, so the args are what the guard checks. Wrapping suspends the guard for the wrapped server only, because a wrap is an explicit statement that this one server is your entire surface, and it warns you when that happens.

Two eras, served per request. The 2026-07-28 revision removes the handshake that defines

the older protocol, so "which era" is a property of each request, and that is how the gateway treats it: era detection per request, no mode switch, no configuration. Legacy answers stay byte-identical to what a pre-0.6.0 client saw; modern decoration only ever appears on modern requests.

Config lives in one folder. Everything mutable lives in the config home, relocatable with `--config-dir` or `TOOLFUNNEL_HOME`, seeded once, never overwritten by updates. One folder to back up, one folder to package, one folder to hand a colleague. The gateway prints the resolved home at every start so there is never any doubt about where state lives.

17. Troubleshooting

The client connects but sees only four tools. That's the design working: the four meta-tools ARE the surface. Call `toolfunnel_list_tools` to see the briefs. If you want specific tools advertised top-level on every turn, promote them (`hot`).

`EADDRINUSE` on `--http` or `--ui`. Something already owns the port. Use `--port 0` for an OS-assigned one, or set `httpPort` / `uiPort` in `toolfunnel.json`.

The UI or HTTP host refuses to start on `--host 0.0.0.0`. Deliberate. The UI is loopback-only by design, full stop. The HTTP host unlocks non-loopback binds only with OAuth enabled (section 13).

A tool runs but returns nothing. Check the script contract (section 6.2): args come from the `TOOLFUNNEL_TOOL_ARGS` environment variable, and the script must print exactly one JSON line to stdout and exit 0. Anything printed to stderr is treated as diagnostics; the result is that single stdout JSON line.

An upstream will not connect. Try the UI's Discover button for a live connect attempt with the error surfaced. Common causes: the command is not on PATH (spell out the full interpreter path), or a path-shaped arg escapes the gateway root and the isolation guard rejected it (vendored upstreams live inside the tree; section 7.1).

On Windows, an upstream spawn fails oddly. Non-`.exe` commands (`npx`, `.cmd` shims) are routed through a cmd.exe wrapper (the Node CVE-2024-27980 workaround). Prefer full paths to real executables where you can. One honest caveat: args containing cmd.exe metacharacters (`& | > <`) can be reinterpreted on that path; those args come from your own config, but know the edge exists.

A hook does not fire. Check the overlay: `hooks/hooks.state.json` wins over the manifest seed, and the shipped examples start disabled. `tf_list { "kind": "hooks" }` shows the live state. Remember matchers are full-anchored: `hash` does not match `hash2`.

Everything disappeared from `tools/list`. Someone hid the meta-tools without promoting replacements (the plain-MCP posture, half-applied). Re-enable them in `tools/tools.state.json` (set the meta-tool names' `hot` back to `true`) or via the UI's surface panel.

Where do I report a bug? GitHub issues, and if it is security-sensitive, GitHub's private vulnerability reporting on the repository. A bug with a fix attached is the fastest route to a merge :)

18. Links

- Repository: https://github.com/Rendeverance/toolfunnel
- npm: https://www.npmjs.com/package/toolfunnel
- Glama listing: https://glama.ai/mcp/servers/Rendeverance/toolfunnel
- Packaging deep-dive: `docs/packaging.md` in the repository
- MCP specification: https://modelcontextprotocol.io